HDOC DAY–10 ACTIVITY

QUESTION 1 – Time Conversion

Write a C program to input time in seconds and convert it into days, hours, minutes, and seconds format.

1. Problem Understanding

- **Input:** Time in seconds.
- **Output:** Equivalent time in days, hours, minutes, and seconds.
- **Formula:** 1 day = 86400 seconds, 1 hour = 3600 seconds, and 1 minute = 60 seconds.

Formula Summary

Quantity	Formula
Days	days = total seconds / 86400
Hours	hours = remaining seconds / 3600
Minutes	minutes = remaining seconds / 60
Seconds	seconds = remaining seconds

2. Standard Test Cases

Test Case	Input	Expected Output	Actual Output	Result (Pass/Fail)
1	90061	1 day, 1 hour, 1 minute, 1 second	1 day, 1 hour, 1 minute, 1 second	Pass
2	86400	1 day, 0 hours, 0 minutes, 0 seconds	1 day, 0 hours, 0 minutes, 0 seconds	Pass
3	3665	0 days, 1 hour, 1 minute, 5 seconds	0 days, 1 hour, 1 minute, 5 seconds	Pass

3. Algorithm

1. Start the program.

2. Declare integer variables for total seconds, days, hours, minutes, and seconds.

3. Read the total time in seconds.

4. Calculate days by dividing total seconds by 86400.

5. Find the remaining seconds using the remainder operator.

6. Calculate hours and update the remaining seconds.

7. Calculate minutes; the final remainder is seconds.

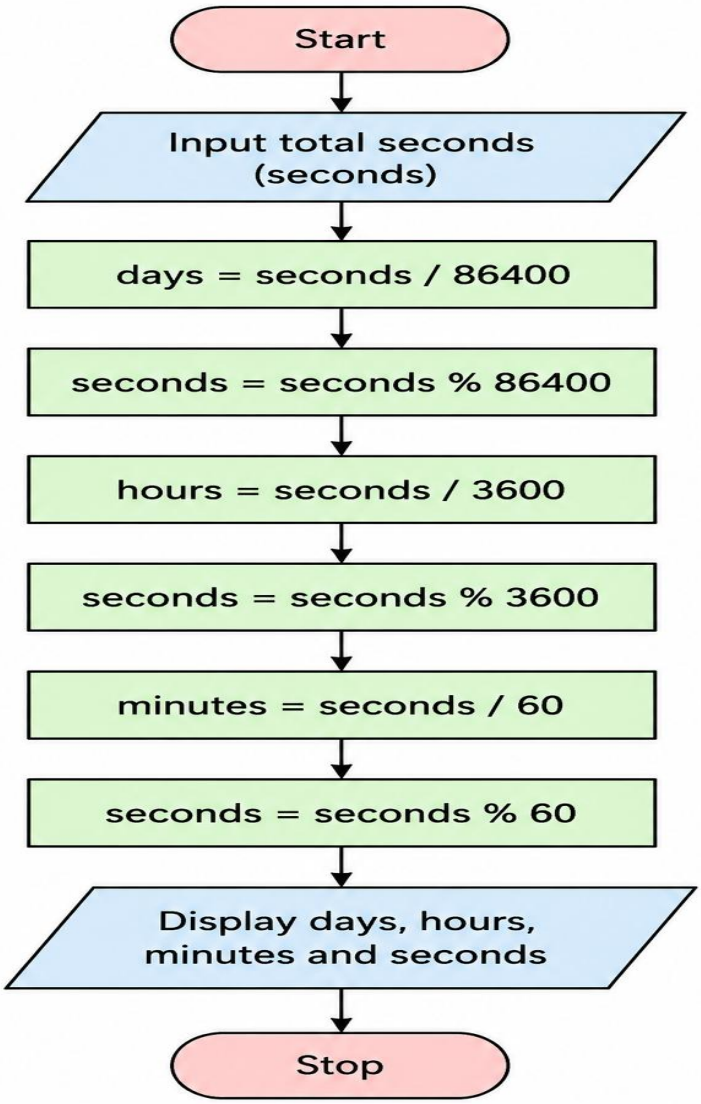
8. Display days, hours, minutes, and seconds.

9. Stop the program.

4. Pseudocode

```
START
INPUT total_seconds
days ← total_seconds / 86400
remaining ← total_seconds % 86400
hours ← remaining / 3600
remaining ← remaining % 3600
minutes ← remaining / 60
seconds ← remaining % 60
OUTPUT days, hours, minutes, seconds
STOP
```

5. Flowchart



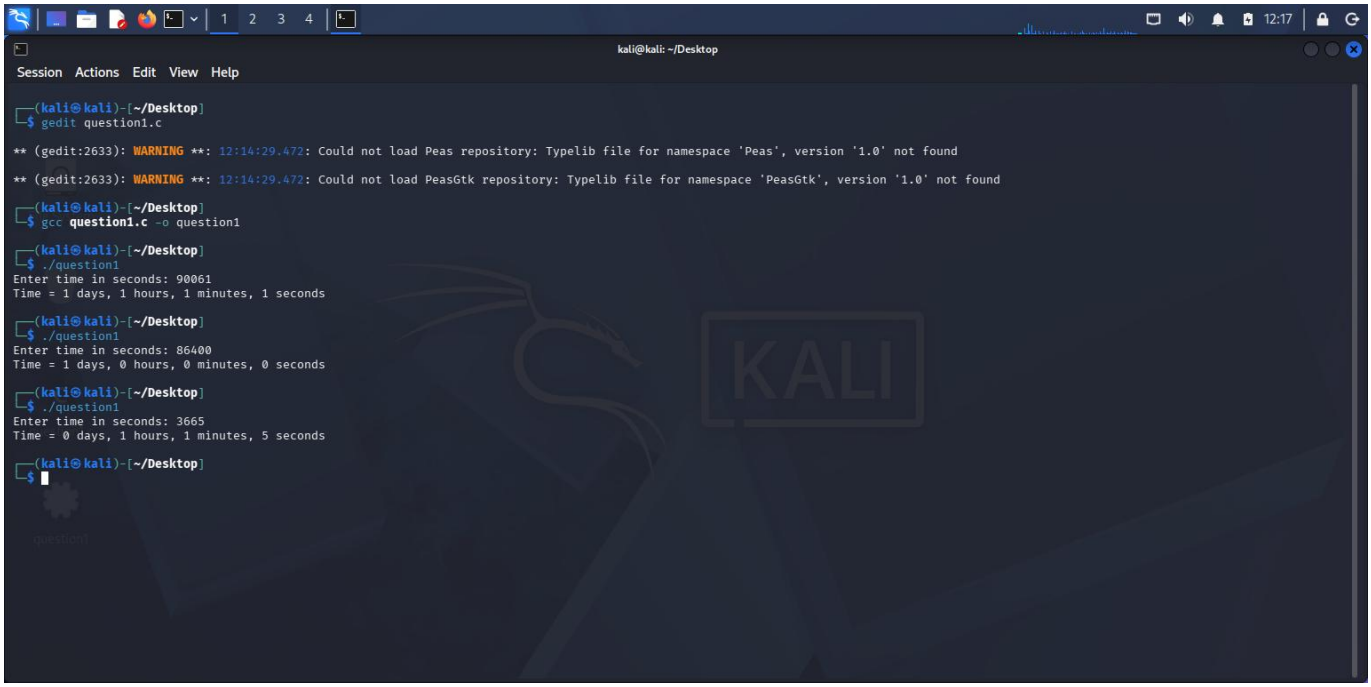
6. C Program

```
#include <stdio.h>
int main(void)
{
int total_seconds, days, hours, minutes, seconds, remaining;
printf("Enter time in seconds: ");
scanf("%d", &total_seconds);
days = total_seconds / 86400;
remaining = total_seconds % 86400;
hours = remaining / 3600;
remaining = remaining % 3600;
minutes = remaining / 60;
seconds = remaining % 60;
printf("Time = %d days, %d hours, %d minutes, %d seconds\n", days, hours, minutes, seconds);
return 0;
}
```

7. Verification of All Test Cases

Test Case	Input	Expected Output	Actual Output	Result (Pass/Fail)
1	90061	1 day, 1 hour, 1 minute, 1 second	1 day, 1 hour, 1 minute, 1 second	Pass
2	86400	1 day, 0 hours, 0 minutes, 0 seconds	1 day, 0 hours, 0 minutes, 0 seconds	Pass
3	3665	0 days, 1 hour, 1 minute, 5 seconds	0 days, 1 hour, 1 minute, 5 seconds	Pass

8. Compilation and Execution Evidence



QUESTION 2 – Leap Year

Write a program to input a year and check whether it is a leap year or not.

1. Problem Understanding

- **Input:** A year as an integer.
- **Output:** Whether the given year is a leap year or not.
- **Condition:** A year is a leap year if it is divisible by 400, or if it is divisible by 4 but not divisible by 100.

Formula Summary

Quantity	Condition
Leap Year	(year % 400 == 0) OR ((year % 4 == 0) AND (year % 100 != 0))

2. Standard Test Cases

Test Case	Input	Expected Output	Actual Output	Result (Pass/Fail)
1	2024	Leap Year	Leap Year	Pass
2	1900	Not a Leap Year	Not a Leap Year	Pass
3	2000	Leap Year	Leap Year	Pass

3. Algorithm

1. Start the program.

2. Declare an integer variable for year.

3. Read the year.

4. Check whether the year is divisible by 400, or divisible by 4 and not divisible by 100.

5. If the condition is true, display that it is a leap year.

6. Otherwise, display that it is not a leap year.

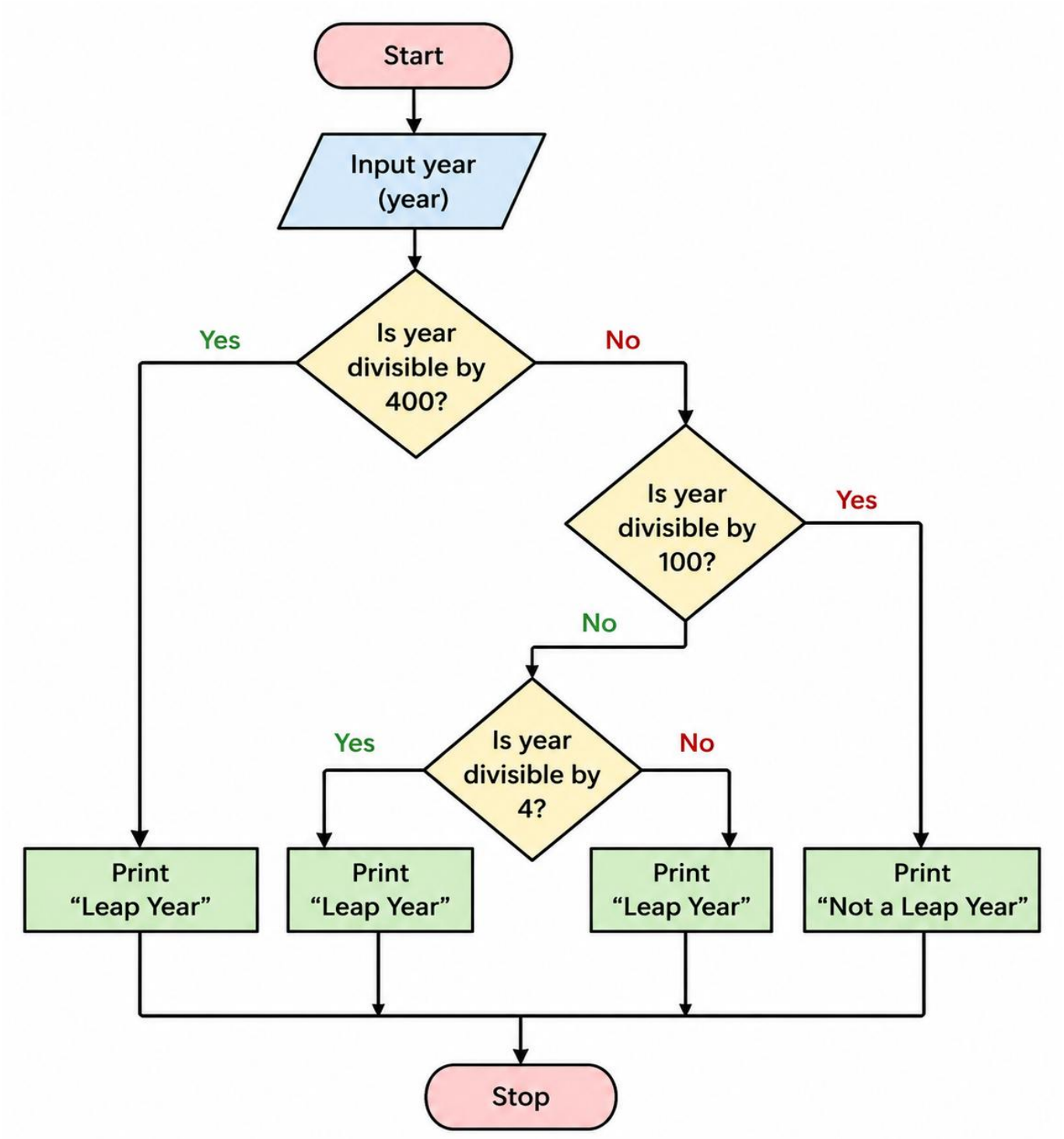
7. Stop the program.

4. Pseudocode

START
INPUT year
IF (year % 400 = 0) OR ((year % 4 = 0) AND (year % 100 ≠ 0)) THEN

```
    OUTPUT "Leap Year"
ELSE
    OUTPUT "Not a Leap Year"
END IF
STOP
```

5. Flowchart



6. C Program

```
#include <stdio.h>
int main(void)
{
int year;
printf("Enter a year: ");
scanf("%d", &year);
if ((year % 400 == 0) || ((year % 4 == 0) && (year % 100 != 0)))
printf("%d is a Leap Year\n", year);
else
printf("%d is Not a Leap Year\n", year);
return 0;
}
```

7. Verification of All Test Cases

Test Case	Input	Expected Output	Actual Output	Result (Pass/Fail)
1	2024	Leap Year	Leap Year	Pass
2	1900	Not a Leap Year	Not a Leap Year	Pass
3	2000	Leap Year	Leap Year	Pass

8. Compilation and Execution Evidence